

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 10
JAVA COLLECTION



Disusun Oleh:
Zakiati Latifa 105224033

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Java Collection Framework (JCF) adalah sebuah arsitektur standar pada Java yang berfungsi untuk menyimpan, mengelola, dan memanipulasi sekumpulan objek secara dinamis. Berbeda dengan struktur data array konvensional yang memiliki batasan ukuran tetap (statis), Collection mengalokasikan memori secara fleksibel sehingga ukurannya dapat bertambah atau berkurang secara otomatis saat program berjalan (runtime). Kerangka kerja ini menyediakan berbagai antarmuka (interface) dengan karakteristik penyimpanan data yang berbeda-beda untuk mendukung efisiensi memori dan kecepatan program.

Dalam implementasi program manajemen gudang ini, digunakan struktur data Map dan Set. Map, yang diimplementasikan melalui kelas HashMap, menyimpan data dalam format pasangan kunci dan nilai (key-value). Kunci yang bersifat unik, seperti idBarang, digunakan sebagai indeks referensi untuk mencari dan memodifikasi objek Barang dengan sangat cepat. Sementara itu, struktur data Set melalui kelas HashSet digunakan untuk menampung data kategori. Sifat utama Set yang tidak mengizinkan adanya duplikasi data memastikan bahwa setiap nama kategori hanya akan dicatat satu kali di dalam sistem, meskipun terdapat banyak barang dengan kategori yang sama.

Selain itu, program ini juga memanfaatkan antarmuka List yang diimplementasikan melalui kelas ArrayList untuk menangani pencatatan data yang terurut. Berbeda dengan Set, struktur data List mengizinkan penyimpanan elemen duplikat dan menyusun datanya secara sekuensial berdasarkan indeks (dimulai dari nol). Karakteristik ini sangat ideal diaplikasikan pada variabel riwayat, karena mampu menyimpan log aktivitas atau catatan transaksi penambahan stok barang secara kronologis dan berurutan sesuai dengan waktu eksekusinya. Penggunaan ketiga struktur data ini secara bersamaan menjadikan pengelolaan informasi dalam program lebih modular, rapi, dan sesuai dengan prinsip Pemrograman Berorientasi Objek (OOP).

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	idBarang	String	Untuk menyimpan idBarang dalam bentuk text
2	namaBarang	String	Untuk menyimpan nama barang dalam bentuk text
3	kategori	String	Untuk menyimpan kategori barang dalam bentuk text

4	stok	int	Untuk menyimpan stok barang dalam bilangan bulat
5	gudang	Map<String, Barang>	Untuk menyimpan sekumpulan objek barang secara dinamis dengan menggunakan idBarang sebagai kata kunci pencarian.
6	kategori	Set<String>	Untuk menampung daftar nama kategori dari barang-barang yang ada, di mana sifat Set akan mencegah adanya duplikasi data kategori.
7	riwayat	List<String>	Untuk menyimpan urutan log riwayat transaksi setiap kali terjadi penambahan stok barang ke dalam sistem.

III. Constructor dan Method

Dalam sistem pengiriman ini menggunakan metode sebagai berikut:

No	Nama Metode	Jenis Metode	Fungsi
1	Barang()	<i>constructor</i>	Untuk menginisialisasi objek baru dari atribut class barang
2	tambahBarangBaru()	<i>Procedural</i>	Untuk menyimpan data objek barang baru ke dalam koleksi penyimpanan utama (HashMap) dan mendaftarkan kategorinya secara unik (HashSet)
3	tambahStok()	<i>Procedural</i>	Untuk menambah jumlah ketersediaan stok pada barang yang sudah ada berdasarkan ID nya, sekaligus mencatat aktivitas tersebut ke dalam riwayat transaksi
4	kurangiStok()	<i>Procedural</i>	Untuk mengurangi jumlah stok barang berdasarkan ID dengan melakukan validasi terlebih dahulu agar sisa stok tidak menjadi negatif.

5	cetakLaporan()	<i>Procedural</i>	Untuk menampilkan rekapitulasi data seluruh barang di dalam sistem, yang mencakup informasi kategori, sisa stok akhir, dan riwayat transaksi
6	Main	<i>Procedural</i>	Untuk menjalankan (mengeksekusi) program

IV. Dokumentasi dan Pembahasan Code

1.

```

1  class Barang {
2      String idBarang;
3      String namaBarang;
4      String kategori;
5      int stok;
6      Barang(String idBarang, String namaBarang, String kategori, int stok)
7      {
8          this.idBarang = idBarang;
9          this.namaBarang = namaBarang;
10         this.kategori = kategori;
11         this.stok = stok;
12     }

```

- Disini saya membuat kelas Barang sebagai blue print dengan atribut idbarang, nama barang, kategori barang, dan stok. Lalu di baris 6 saya membuat konstruktor untuk menginisialisasi dari atribut atribut tersebut.
2. Selanjutnya membuat kelas SistemGudang yang bertugas mengelola data barang dengan menggunakan java collection.

```

1  import java.util.*;
2  public class SistemGudang {
3      Map<String, Barang> gudang = new HashMap<>();
4      Set<String> kategori = new HashSet<>();
5      List<String> riwayat = new ArrayList<>();
6      public void tambahBarangBaru(Barang barang) {
7          gudang.put(barang.idBarang, barang);
8          kategori.add(barang.kategori);
9      }
10     public void tambahStok(String idBarang, int jumlah) {
11         riwayat.add("Barang masuk: " + idBarang + " ditambah " + jumlah + " unit.");
12         if (gudang.containsKey(idBarang)) {
13             Barang barang = gudang.get(idBarang);
14             barang.stok += jumlah;
15         } else {
16             System.out.println("Barang dengan ID " + idBarang + " tidak ditemukan.");
17         }
18     }
19     public void kurangiStok(String idBarang, int jumlah) {
20         if (gudang.containsKey(idBarang)) {
21             Barang barang = gudang.get(idBarang);
22             if (barang.stok >= jumlah) {
23                 barang.stok -= jumlah;
24             } else {
25                 System.out.println("Stok tidak cukup untuk mengurangi " + jumlah + " dari " + barang.namaBarang);
26             }
27         } else {
28             System.out.println("Barang dengan ID " + idBarang + " tidak ditemukan.");
29         }
30     }
31     public void cetakLaporan() {
32         for (String i : gudang.keySet()) {
33             Barang barang = gudang.get(i);
34             System.out.println("Kategori: " + barang.kategori + ", Sisa Stok: " + barang.stok + ", " + riwayat.get(index: 0));
35         }
36     }
37 }

```

- Baris 1 terdapat fitur dari java collection.
- Baris 3 menyimpan objek barang menggunakan HashMap Tipe String bertindak sebagai key (kunci pencarian, yang nantinya diisi dengan idBarang), dan Barang bertindak sebagai value (nilai datanya). Penggunaan Map sangat efisien untuk mencari data barang secara spesifik berdasarkan ID-nya.
- Baris 4 menyimpan daftar nama kategori dengan menggunakan HashSet. Sifat utama Set adalah tidak menerima data duplikat. Jadi, jika ada banyak barang dengan kategori Furniture, kata Furniture hanya akan tersimpan satu kali di dalam Set ini.
- Baris 5 untuk menyimpan catatan riwayat penambahana stok menggunakan ArrayList. List ini menjaga urutan data sesuai dengan waktu dimasukkannya data tersebut.
- Baris 6-9 fungsi mendaftarkan barangg baru kedalam sistem. Syntax Put digunakan untuk memasukkan baranng kedalam HashMap gudang dengan idBarang. Syntax Add digunakan untuk mendaftarkan kategori barang kedalam HashSet.
- Baris 10-18 metode untuk menambahkan jumlah stok barang yang udah ada. Sistem akan catat aktivitas ini ke dalam ArrayList kemudia mebuat percabangan untuk mengecek apakah ID barang yang dicari benar benar ada di dalam sistem. Jika ada, sistem akan mengambil objek barang tersebut dan menambahkan jumlah stoknya (barang.stok += jumlah;). Jika tidak ada, sistem akan mencetak pesan bahwa barang tidak ditemukan.

- Baris 19-30 metode mengurangi stok dengan logika sistem mengecek ketersediaan ID barang terlebih dahulu. Jika barang ditemukan, terdapat pengecekan kedua: if (barang.stok >= jumlah). Ini memastikan sisa stok saat ini lebih besar atau sama dengan jumlah yang ingin ditarik. Jika stok cukup, maka stok akan dikurangi (barang.stok -= jumlah;). Jika stok tidak cukup, atau ID barang tidak ditemukan, sistem akan mencetak pesan error atau peringatan yang sesuai.
- Baris 31-36 untuk mencetak laporan. Menggunakan perulangan for untuk menelusuri seluruh key (idBarang) yang ada di dalam HashMap gudang.keySet(). Pada setiap perulangan, sistem mengambil objek Barang terkait dan mencetak kategori serta sisa stoknya.
- Baris 34 terdapat pemanggilan riwayat.get(0). Perintah ini akan selalu mengambil data indeks ke-0 (data pertama) dari ArrayList riwayat. Akibatnya, saat laporan dicetak untuk beberapa barang, pesan riwayat yang muncul akan selalu tulisan riwayat yang sama (riwayat paling awal) secara berulang-ulang, bukan seluruh riwayat transaksi.

3. Lalu saya membuat kelas main untuk mensimulasikan dari kelas sebelumnya

```

1  public class Main {
2      Run | Debug
3      public static void main(String[] args) {
4          SistemGudang gudang = new SistemGudang();
5          Barang barang1 = new Barang(idBarang: "B001", namaBarang: "Laptop", kategori: "Elektronik", stok: 10);
6          Barang barang2 = new Barang(idBarang: "B002", namaBarang: "Meja", kategori: "Furniture", stok: 20);
7          Barang barang3 = new Barang(idBarang: "B003", namaBarang: "Kursi", kategori: "Furniture", stok: 15);
8          gudang.tambahBarangBaru(barang1);
9          gudang.tambahBarangBaru(barang2);
10         gudang.tambahBarangBaru(barang3);
11         gudang.tambahStok(idBarang: "B001", jumlah: 5);
12         gudang.kurangiStok(idBarang: "B002", jumlah: 2);
13         gudang.kurangiStok(idBarang: "B003", jumlah: 20);
14         gudang.cetakLaporan();
15     }
16 }

```

- Baris 3: SistemGudang gudang = new SistemGudang(); Kode ini membuat sebuah instance atau objek baru dari kelas SistemGudang dan menyimpannya dalam variabel bernama gudang. Objek inilah yang nantinya akan menampung seluruh data Collection (HashMap, HashSet, ArrayList) selama program berjalan.
- Baris 4-6 membuat tiga objek barang yang berbeda menggunakan constructor dari kelas Barang
- Baris 7-9 merupakan metode gudang.tambahBarangBaru() dipanggil untuk memasukkan barang1, barang2, dan barang3 ke dalam HashMap dan HashSet yang ada di dalam objek gudang.
- Baris 10-12 merupakan bagian menguji logika metode penambahan dan pengurangan stok.

- Baris 10 (tambahStok): Menambahkan 5 unit ke barang "B001" (Laptop). Stok laptop yang awalnya 10 akan menjadi 15.
 - Baris 11 (kurangiStok): Mengurangi 2 unit dari barang "B002" (Meja). Stok meja yang awalnya 20 akan menjadi 18.
 - Baris 12 (kurangiStok): Mencoba mengurangi 20 unit dari barang "B003" (Kursi). Ini adalah pengujian logika validasi. Karena stok awal kursi hanya 15, maka pengurangan ini tidak akan dilakukan dan program akan mengeksekusi blok else untuk mencetak peringatan ("Stok tidak cukup").
 - Baris 13: gudang.cetakLaporan(); memanggil metode untuk menampilkan kondisi akhir dari sisa stok semua barang beserta kategori dan log riwayat yang tersimpan di dalam sistem setelah serangkaian transaksi di atas dilakukan.
4. Output yang diinginkan jika sudah menjalankan program ini seperti dibawah ini

```
Stok tidak cukup untuk mengurangi 20 dari Kursi
Kategori: Elektronik, Sisa Stok: 15, Barang masuk: B001 ditambah 5 unit.
Kategori: Furniture, Sisa Stok: 18, Barang masuk: B001 ditambah 5 unit.
Kategori: Furniture, Sisa Stok: 15, Barang masuk: B001 ditambah 5 unit.
```

IV. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan Java Collection Framework sangat efektif dalam mengatasi keterbatasan ukuran memori statis pada array konvensional untuk membangun sistem berskala dinamis. Melalui implementasi program manajemen gudang, struktur data HashMap terbukti sangat efisien dalam memetakan dan mempercepat pencarian objek barang berbasis ID (key), sedangkan HashSet berperan krusial dalam mencegah terjadinya duplikasi data pada penyimpanan kategori. Selain itu, pemanfaatan ArrayList menunjukkan kinerja yang ideal dalam menjaga urutan kronologis pencatatan log riwayat transaksi stok barang. Secara keseluruhan, integrasi ketiga struktur data Collection ini menjadikan pengelolaan data di dalam program lebih terstruktur, memori lebih fleksibel, dan sangat sejalan dengan prinsip-prinsip utama dalam Pemrograman Berorientasi Objek (OOP).